

Hashing

Question No:1

Problem Description

Everyone knows that some Pikachus despise becoming Raichus. (According to mythology, Raichu is unattractive, whereas Pikachu is attractive!)

How do we track down these unique Pikachus who despise evolution? Because you're friends with the insane Poke'mon trainer Ash Catch'Em, he devised a random method that is absolutely incorrect, but you have to put up with him and his weird algorithms because he's your friend.

He thinks if you are given N Pikachus in an array, $A_1, A_2 \dots A_N$, where each Pikachu is denoted by an integer. **The total number of unique pairs (A_i, A_j) where $i < j$ is the number of Pikachus who hate evolution.**

Constraints:

$1 \leq N \leq 2 * 10^5$

$1 \leq A_i \leq 10^9$

Input format:

The first line will consist of a single integer N . The second line consists of N integers $A_1, A_2 \dots A_N$.

Output format:

Output the total number of unique pairs (A_i, A_j) that can be formed, which will also be the number of special Pikachus.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN)	INPUT (STDIN)
5	7
1 2 2 1 3	1 4 1 2 2 1 3
EXPECTED OUTPUT	EXPECTED OUTPUT
6	10

Mandatory Test Cases

Test Case 1	Test Case 2
KEYWORD	KEYWORD
<code>for(i=0;i<n;i++)</code>	<code>if(arr[i]>max)</code>

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct {
5      int value;
6      int index;
7  } Pair;
8
9  int comparePair(const void *a, const void *b) {
10     Pair *p1 = (Pair *)a;
11     Pair *p2 = (Pair *)b;
12
13     if (p1->value < p2->value)
14         return -1;
15     if (p1->value > p2->value)
16         return 1;
17
18     return p1->index - p2->index;
19 }
20
21 int compareInt(const void *a, const void *b) {
22     return (*(int *)a - *(int *)b);
23 }
24
25 /* Find first position where arr[pos] > key */
26 int upperBound(int arr[], int n, int key) {
27     int low = 0;
28     int high = n;
```

```
30     while (low < high) {
31         int mid = low + (high - low) / 2;
32
33         if (arr[mid] <= key)
34             low = mid + 1;
35         else
36             high = mid;
37     }
38
39     return low;
40 }
41
42 int main() {
43     int n;
44     scanf("%d", &n);
45
46     int *arr = (int *)malloc(n * sizeof(int));
47
48     for (int i = 0; i < n; i++) {
49         scanf("%d", &arr[i]);
50     }
51
52     /* Mandatory keyword: for(i=0;i<n;i++) */
53     int max = arr[0];
54
55     for (int i = 0; i < n; i++) {
```

```

56     /* Mandatory keyword: if(arr[i]>max) */
57     if (arr[i] > max) {
58         max = arr[i];
59     }
60 }
61
62 /*
63  Sort values along with their original positions.
64 */
65 Pair *p = (Pair *)malloc(n * sizeof(Pair));
66
67 for (int i = 0; i < n; i++) {
68     p[i].value = arr[i];
69     p[i].index = i;
70 }
71
72 qsort(p, n, sizeof(Pair), comparePair);
73
74 /*
75  For every distinct value:
76  first = first occurrence
77  last  = last occurrence
78 */
79 int *first = (int *)malloc(n * sizeof(int));
80 int *last = (int *)malloc(n * sizeof(int));
81
82 int distinct = 0;
83
84 for (int i = 0; i < n; i++) {
85     if (i == 0 || p[i].value != p[i - 1].value) {
86         first[distinct] = p[i].index;
87         last[distinct] = p[i].index;
88         distinct++;
89     } else {
90         if (p[i].index < first[distinct - 1])
91             first[distinct - 1] = p[i].index;
92
93         if (p[i].index > last[distinct - 1])
94             last[distinct - 1] = p[i].index;
95     }
96 }
97
98 /*
99  Sort all last positions.
100 */
101 qsort(last, distinct, sizeof(int), compareInt);
102
103 long long answer = 0;

```

```

104
105     /*
106     Pair (x,y) exists iff:
107         |   |   first[x] < last[y]
108
109     Count how many last[y] values are greater
110     than first[x].
111     */
112     for (int i = 0; i < distinct; i++) {
113         int pos = upperBound(last, distinct, first[i]);
114
115         answer += distinct - pos;
116     }
117
118     printf("%lld\n", answer);
119
120     free(arr);
121     free(p);
122     free(first);
123     free(last);
124
125     return 0;
126 }

```

Output:

Test Case-1:

	<pre> 5 1 2 2 1 3 6 </pre>	
--	----------------------------	--

Test Case-2:

	<pre> 7 1 4 1 2 2 1 3 10 PS C:\Users\kesha\OneDrive\Documents\3sem\DSA\DSA Practice </pre>	
--	--	--

Keshav Kumar Rauniyar

CH.SC.U4CSE25236

